Message guard demo, Lecture 01 — the five judgments on version A0

The demonstration that runs through Lecture 01 of the ASE principles module. The repository is version A0 of the message guard, complete and green: one stage, `filter`, with its specifications, tests, rules, and reports. Nothing is built during class. The class is walked through the artifacts in the order of the lecture notes, the gate is run, the three stories are walked, and two judgments are broken to show which part of the repository notices. The slides show the same files and the same command output, so the room sees on the screen what the deck has already shown them; the point of running it live is that the output is produced in front of them, not pasted.

What governs the whole script. Everything except the optional last segment is deterministic: shell commands over a fixed snapshot. Rehearse once anyway, because the gate run depends on `uv` having fetched the tools, and because the sabotage segments edit a file and must be restored cleanly. The optional agent segment can go differently from run to run; it has a fallback capture.

Slide references below are by slide title.

What A0 contains, and what the demonstration relies on

`v0/` holds the artifacts by category (the table in `v0/README.md`): the two specifications, the code and the tests, the rules file, four reports, the technology files, and two support scripts. Three facts about the snapshot are what the segments turn on; do not point them out before their segment.

- **The gate is green and the ledger has five established rows.** Segment 4 shows this; Segments 5 and 6 break it.
- **REQ-2 says "unchanged" and the test decides it by equality.** The specification-quality segment asks the room whether that is the only reading. It is the one place where a reading is visibly a judgment.
- **Nothing in A0 can tell that Restricted messages were meant to be stopped.** Segment 6 constructs that scenario. There is no command for it; the whole point is that no command exists.

There are no seeded gaps in A0's specifications. Gaps placed on purpose are the A1 demonstration, for the lecture on the audit.

Before class — setup checklist

1. Make the demo repository from the snapshot, with one commit so that every sabotage can be undone with `git checkout`:

   ```
   rm -rf ~/message-guard-demo
   cp -R <path-to-this-folder>/v0 ~/message-guard-demo
   cd ~/message-guard-demo
   git init && git add -A && git commit -q -m "v0: message guard, version A0"
   ```

2. Fetch the tools and confirm the gate is green. This also regenerates `reports/conformance.md` with today's timestamp:

   ```
   uv sync
   tools/gate.sh; echo "exit $?"
   git status --short        # only reports/conformance.md should show, if anything
   git commit -qam "regenerate the conformance report" 2>/dev/null; true
   ```

   The run must end with `10 passed`, `Total coverage: 100.00%`, and `conformance: every test clause has passing evidence`. If `uv` is not installed, install it first; the demo has no other dependency.

3. Rehearse Segment 5 once (the sabotage and its restore) and Segment 7 if you will run it, then reset with step 1.

4. Two terminals with a large font, both in `~/message-guard-demo`: one for `cat`, `git`, and the scripts; one kept clean for the gate runs so their output is not scrolled away. An editor open on `specs/conops.md`, `specs/requirements.md`, and `reports/ledger.md` in tabs, for the moments when a file is easier to point at than to `cat`.
5. If Segment 7 will run: start Claude Code in `~/message-guard-demo` in its default permission mode before class and confirm it read `CLAUDE.md`. Keep the session idle until the segment. Capture the rehearsal's answer at the `[fallback capture]` marker.
6. Have the two sabotage commands ready to paste (Segment 5 gives them), so no time is spent typing a `sed` expression in front of the room.

Segment 1 — The question the code cannot answer (about 5 minutes)

Slides: *The example: a message guard*; *The question the code cannot answer*; *Version A0, by category.*

Do (shell): `cat guard/filter.py`.

Say: this is the whole file. Is it right? The room will say yes. Ask what it was supposed to do; someone will say "drop Critical". Ask how they know that; the honest answer is the comment and the name of the constant. Then: it would look exactly like this if the security officer had wanted Restricted messages stopped as well, and it would be wrong. The code cannot say what was asked for, what was promised, or whether the promise was checked.

Do (shell):

```
ls specs guard tests process reports tools
```

Say: read the directories against the category table on the slide: the specifications, the realization (code and tests), the rules, the reports, the support scripts. There is no file for intent, and that is deliberate; it is the first thing to come back to.

Segment 2 — The specifications (about 12 minutes)

Slides: *Intent in A0: the purpose statement*; *The specifications in A0: a family of two*; *The clause is the unit*; *Mechanical clauses and judgment clauses*; *Specification quality on A0.*

Do (editor or shell): `cat specs/conops.md`. Read the purpose statement aloud; it is two short paragraphs.

Say: the nearest written thing to intent. The stakeholders revised it until they agreed; it is audited like any clause; it is not a transcript of what anyone said. How it was obtained is the next lecture. Scroll to the Terms, the two user categories, and the three stories; stop on US-1 and read its header line: identifier, method, user category. Every story is marked *walkthrough (human)*: a person exercises the guard and judges.

Do (shell): `cat specs/requirements.md`.

Say: four requirements, each with an identifier, a method, and what it derives from. Read REQ-1's header line and point at *Method: test* and *Derived from: US-3*: a program will decide this clause, and it traces back to the officer's story. The method is on the clause rather than in the verifier's head because the choice has consequences other people need to see: a test clause is checked at every commit by a program nobody has to remember to run; a walkthrough clause costs a person's attention each time.

Ask the room: REQ-2 says a Restricted message passes "unchanged". What does unchanged mean? Someone will say "same level, same payload"; someone may say "the same object". The test decides it by equality. Is that the only reading? Point at the Terms section, where *pass* is defined, and at REQ-0, which says a message has exactly two fields. Whether that settles it is the kind of question the audit in the next lecture is built to ask; today, a person read the seven clauses together and recorded the reading in the ledger (rule SQ-1).

Expected: no surprises; these are static files. Keep this segment moving; the slides already carry the five quality properties, so name them against the files rather than reading the table again.

Segment 3 — The realization and the tests (about 6 minutes)

Slides: *The realization*; *Tests as executable specification*; *Realization quality (RQ) in A0.*

Do (shell): `cat guard/message.py`, then `cat tests/test_filter.py`.

Say: the enumeration with exactly three members and the frozen dataclass are REQ-0 made concrete; the type checks in `__post_init__` are why branch coverage is not trivial. In the tests, point at `@pytest.mark.req("REQ-1")`: the citation. It serves three readers: a program computes which clauses have tests; a person who sees the test fail knows which clause is in question; an agent writing a test must say what it is evidence for. A test that cites nothing is not allowed in this suite.

Do (shell):

```
uv run ruff check .
uv run pytest --collect-only -q --no-cov | tail -1
```

Say: realization quality, decided by two programs: the code lints and the suite collects, ten tests. This is kept separate from conformance so that a conformance result is about code that runs and a suite that ran.

Segment 4 — The gate, the evidence, and the walkthroughs (about 12 minutes)

Slides: *Conformance is four conditions*; *The evidence*; *Who decided*; *Validation in A0: the sign-off*; *The gate.*

Do (clean terminal): `tools/gate.sh`.

Say, while it runs: one command decides realization quality and the three mechanical conditions of conformance. Read the output bottom-up: `10 passed`; the coverage table with every row at 100 percent and the line that the required coverage was reached; the plugin's line that every test clause has passing evidence. Three programs, three conditions.

Do (shell): `cat reports/conformance.md`.

Say: one row per clause, read from the specifications by the plugin, not from a hand-kept list. The four requirements show their tests and *pass*; the three stories show *walkthrough: see reports/walkthrough.md*, because no test decides them. Requirements coverage, 4 of 4. Unknown identifiers, none.

Do (shell): the three walkthroughs, live, in the roles the stories name:

```
uv run python tools/walk.py Public 42
uv run python tools/walk.py Restricted 7
uv run python tools/walk.py Critical 100
uv run python tools/walk.py Critical 0
```

Say: the fourth condition. Announce the role before each command: "as the message sender, US-1"; "as the security officer, US-3". The outputs are the *Observed* column of the walkthrough record.

Do (shell): `cat reports/walkthrough.md`.

Say: the table the room just reproduced, and below it the sign-off: who, in which role, when, on which version. The walkthrough did two jobs at once. When the officer walked US-3 and saw `drop`, two questions were answered: does the system behave as the story says, and is the story what the officer wants. The first is conformance of a judgment clause; the second is validation, and it is the one judgment no program can make.

Expected: the gate takes under five seconds after the first `uv sync`. **If it goes differently:** a non-zero exit here means the setup checklist was not completed; run `git status` and `git checkout -- .`, then `uv sync` and retry. The outputs of `walk.py` are deterministic.

Segment 5 — Breaking it: a wrong realization (about 6 minutes)

Slide: *Breaking it: a wrong realization.*

Do (shell): change the condition so that Critical passes and Restricted is dropped, then run the gate:

```
sed -i '' 's/SecurityLevel.CRITICAL:/SecurityLevel.RESTRICTED:/' guard/filter.py
git diff --stat
tools/gate.sh; echo "exit $?"
```

(On Linux the `sed` flag is `-i` without the empty string.)

Say: four tests fail: the three cases for REQ-1 and the one for REQ-2. Read the plugin's lines: `REQ-1: FAIL`, `REQ-2: FAIL`. Point at the coverage table: still 100 percent. Coverage says which code ran, not whether it was right. The row that turned red is CONF and the verifier that caught it is the test suite; SQ, VAL, and TR are untouched, because no specification changed and every identifier still resolves.

Do (shell): `grep -E "REQ-[12]" reports/conformance.md` to show the two FAIL rows in the report, then restore:

```
git checkout -- guard/filter.py
tools/gate.sh > /dev/null; echo "exit $?"
```

Say: a failed check reports a fact about the pair: the specification and the realization disagree. It does not say which side is wrong. Here the code moved back. In other cases the specification is amended, and the decision is made by a person and recorded; the dev log is A0's record, and the fuller machinery arrives in version A3.

`[fallback capture]` the gate output with the two FAIL lines and the 100 percent coverage table, in case the live run is interrupted.

Segment 6 — Breaking it: a wrong system with a green ledger (about 6 minutes)

Slide: *Breaking it: a wrong system with a green ledger.*

Say: suppose the security officer had meant that Restricted messages must be stopped as well, and the person who wrote the concept of operations did not understand this. Look at the repository. US-2 and REQ-2 agree with each other. Every requirement has a passing test. Branch coverage is complete. Every identifier resolves. Four rows established on sound evidence, and the fifth too if the officer signs without reading.

Do (shell): `cat reports/ledger.md`, and point at the five established rows; then, as the officer:

```
uv run python tools/walk.py Restricted 7
```

Say: `pass: Restricted 7`. This is the moment. The officer who wanted Restricted stopped says "that is not what I want", or reads the concept of operations and declines to sign. Nothing else in the repository can detect it, because every artifact was derived from the same misunderstanding. That is why validation is its own row, and why its evidence is a person's recorded decision and not a tool's output. There is no command to run for this sabotage; that is its point.

Expected: this segment runs on the restored repository; confirm the gate was green at the end of Segment 5 before starting it.

Segment 7 — The ledger, the rules, and the log (about 6 minutes)

Slides: *The ledger at v0*; *The development rules: eight, in one form*; *Why the rules are a file, and where the effort goes.*

Do (editor): `reports/ledger.md`, the CONF row.

Say: four parts: the status, the evidence named precisely enough to find, the provenance (which verifier and version, which artifacts at which version, which clauses), and what makes the row stale. Provenance is what separates evidence from a description of evidence: "pytest 9.1.1 exited 0 at v0" can be rerun; "all tests pass" in an agent's completion message cannot.

Do (shell): `grep -A 11 "^## CONF-1" process/rules.md`.

Say: one rule in the template: style, the judgment it maintains, when, the statement, who decides, the evidence, what happens on failure, one example from this system. A rule with no decider is a wish; a rule with no failure behavior leaves a red result with nowhere to go. Eight rules in A0, two styles.

Do (shell): `cat CLAUDE.md`, then `head -20 reports/dev-log.md`.

Say: the rules are a file because the agent reads it: `CLAUDE.md` tells an agent to read the rules and the specifications before acting, and from then on the person and the agent follow the same written process. The dev log is the record of what each task did; the ledger is the record of where things stand.

Do (shell), the closing figure:

```
cat guard/*.py | grep -c .
cat specs/*.md process/rules.md reports/*.md | wc -l
```

Say: under thirty lines of code; more than three hundred lines of specifications, rules, and reports. Not a property of the small example: when an agent writes the code, the person's effort goes into saying what is wanted, deciding how it will be checked, and reading the evidence.

Segment 8 (optional) — Ask an agent (about 5 minutes)

Slide: *Why the rules are a file, and where the effort goes* (the callout).

Run only if the session from the setup checklist is ready and the time is there; the lecture does not depend on it.

Do (agent): in the idle Claude Code session, type:

> What does this system do, and how is it known that the code does it? Answer from the repository, and name the files you used.

Say, before the answer arrives: the useful thing to watch is where it looks.

Expected: an answer that states the behavior from `specs/conops.md` and `specs/requirements.md`, says the code conforms because the tests cite the requirements and pass, cites `reports/conformance.md` or `reports/ledger.md`, and names the walkthrough record for the stories. Point out that this is an answer a person could check, file by file.

If it goes differently: if the agent answers from `guard/filter.py` and describes what the code does, say so: that is the agent's opinion of the code, and it is exactly what the ledger exists to replace. Ask it a second question, "Which clause of the specifications does each test cite?", which forces it to the markers and the report. If the session is slow or the answer rambles, show the `[fallback capture]` from rehearsal and move on; the lesson is the same.

End of the demonstration

Reset for the next run by repeating step 1 of the setup checklist. The two sabotages for the lecture on testing (a deleted test; an unexercised branch) and the dangling-marker sabotage are listed in `v0/README.md` under "Breaking it" and are not run today.

Exercise 1, from the lecture notes, asks students to add a requirement that a negative payload is dropped and to carry it through the same artifacts; point them at `v0/README.md` for how to run the gate on their own copy.

Timing

Segment	Minutes
1 The question the code cannot answer	5
2 The specifications	12
3 The realization and the tests	6
4 The gate, the evidence, the walkthroughs	12
5 Breaking it: a wrong realization	6
6 Breaking it: a wrong system	6
7 The ledger, the rules, the log	6
8 Ask an agent (optional)	5

Fifty-three minutes with the optional segment, forty-eight without, inside a seventy-five-minute meeting that also carries the purpose, the picture, and the learning objectives from the deck. If time runs short, Segment 3's two commands and Segment 7's closing figure are the first cuts; Segments 5 and 6 are not.